PROPOSTA DE ARQUITETURA

Plataforma de Gestão do T800

Dashboard, ROS2, desenvolvimento paralelo, deploy seguro e acesso remoto

Plataforma	EngineAI T800
Ambiente	64-bit / ROS2 Humble
Escopo inicial	1 robô, preparado para expansão
Documento	Versão 0.1 - 25/09/2026

> **Objetivo do documento**
> Organizar uma arquitetura que permita operar o dashboard do T800 de qualquer lugar, desenvolver funcionalidades em paralelo sem conflitos constantes, atualizar módulos com baixo impacto, iniciar tudo automaticamente com o robô e reduzir a superfície de ataque.

Documento de discussão técnica - base para alinhar a equipe antes da implementação.

1. Resumo executivo

A proposta é evoluir o dashboard atual para uma plataforma modular, em que o T800 deixa de depender de um projeto monolítico executado no computador de um desenvolvedor. O dashboard passa a ser apenas uma das interfaces da plataforma; ROS2, telemetria, comandos, visão, deploy, autenticação e logs ficam separados em serviços com responsabilidades claras.

> **Decisão central**
> Não expor ROS2 diretamente na internet. O robô mantém ROS2 localmente e um Robot Agent controla o que pode sair ou entrar. O acesso remoto administrativo ocorre por rede privada criptografada, e o dashboard usa HTTPS com autenticação forte.

Problemas que a nova arquitetura deve resolver

Problema	Impacto
Dependência local	O dashboard atual está muito vinculado ao Mac/ambiente de um integrante.
Acoplamento	Uma alteração em uma parte do sistema tende a derrubar ou exigir reinício de outras partes.
Conflito entre testes	Testes de dois desenvolvedores e o dashboard oficial não convivem bem no mesmo robô.
Deploy frágil	Atualizar código em produção exige mexer diretamente no ambiente que está funcionando.
Acesso limitado	O dashboard só é acessível quando o dispositivo está na mesma rede do robô.
Operação manual	Serviços precisam ser iniciados ou recuperados manualmente.
Segurança	Expor portas, ROS2 ou serviços internos diretamente aumentaria desnecessariamente a superfície de ataque.
Conhecimento disperso	Código e procedimentos em vários computadores dificultam colaboração, rollback e onboarding.

Resultado esperado

- Dashboard acessível por celular ou computador, de qualquer local autorizado.
- Inicialização automática com o robô e recuperação automática de serviços que falharem.
- Módulos independentes: atualizar visão não deve derrubar telemetria; atualizar dashboard não deve reiniciar ROS2.
- Dois desenvolvedores podem testar trabalhos distintos simultaneamente, com isolamento lógico e sem substituir a versão oficial.
- Produção recebe apenas versões aprovadas, com health check e rollback simples.
- ROS2 continua disponível integralmente no robô, mas a internet enxerga somente interfaces explicitamente autorizadas.

2. Princípios da arquitetura

Modular por responsabilidade - Cada serviço faz uma coisa bem definida e pode ser atualizado/reiniciado separadamente.

Produção não é laboratório - Código experimental nunca substitui diretamente o serviço oficial; ele roda em ambiente TEST.

Uma única porta para o hardware - Comandos que podem movimentar o robô passam por um Command Gateway com prioridade e auditoria.

Conexões saem do robô - Evitar publicar portas do T800 na internet. O robô inicia conexões autenticadas para os serviços externos.

Observabilidade desde o início - Logs, versões, saúde dos serviços, CPU, memória, temperatura, bateria e eventos de deploy ficam visíveis.

Rollback é requisito - Toda atualização deve ter uma versão anterior conhecida e recuperável.

Configuração como código - Configurações, compose, scripts, documentação e políticas ficam no repositório, não apenas na memória da equipe.

Menor privilégio - Usuários, serviços e tokens recebem apenas as permissões necessárias.

> **Observação importante sobre segurança**
> Não existe arquitetura com garantia realista de “zero vulnerabilidades”. O objetivo técnico deve ser reduzir superfície de ataque, segmentar privilégios, impedir exposição desnecessária, registrar ações e tornar correções/atualizações rápidas e reversíveis.

3. Arquitetura proposta

A plataforma é dividida em três zonas: interface remota, control plane e software do robô. O T800 continua executando ROS2 Humble localmente. Um Robot Agent faz descoberta do grafo ROS2, coleta telemetria, transmite dados autorizados e recebe comandos validados.

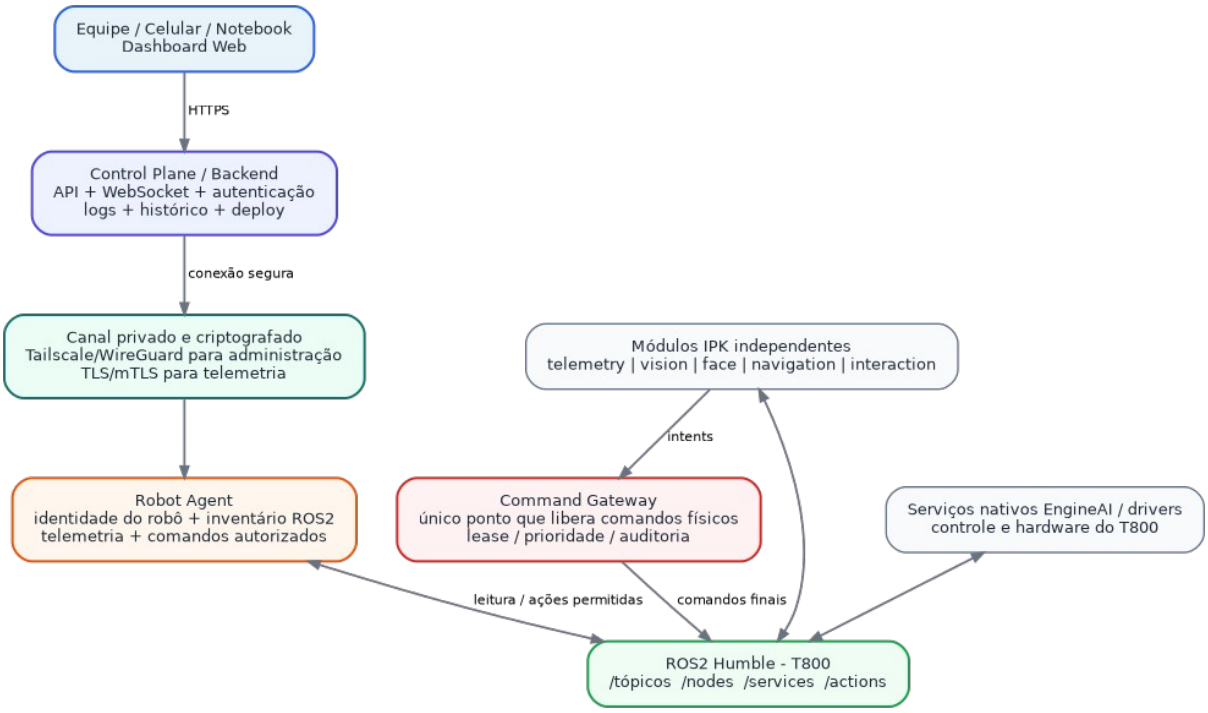


Figura 1 - Arquitetura lógica proposta para o T800.

Responsabilidade de cada camada

Componente	Função
Dashboard Web	Visualização e operação. Não conversa diretamente com DDS/ROS2. Usa API/WebSocket do backend.
Control Plane	Autenticação, autorização, inventário de robôs, histórico, logs, telemetria, comandos e gestão de versões.
Canal privado	Acesso administrativo via VPN/overlay privado; sem expor SSH, DDS ou portas internas ao público.
Robot Agent	Identidade do T800, health, inventário ROS2, ponte de telemetria e execução de ações permitidas.
Command Gateway	Arbitra comandos físicos e impede que PROD e TEST publiquem simultaneamente na mesma saída de controle.
Módulos IPK	Visão, reconhecimento facial, navegação, interação, streaming etc. executados como serviços independentes.
ROS2 Humble	Barramento local do robô. Todos os tópicos continuam disponíveis internamente; dados pesados saem apenas sob demanda.
EngineAI / drivers	Serviços nativos e integração de hardware. Devem ser isolados de alterações desnecessárias da plataforma IPK.

4. Como fazer PROD e testes coexistirem no mesmo T800

Este é o ponto que mais resolve o problema atual. A proposta não é simplesmente “subir cópias do mesmo app”. Cada execução deve ter identidade própria, namespace próprio e limites claros sobre o que pode publicar.

4.1 Namespaces e nomes únicos

```
/ipk/prod/vision
/ipk/prod/face
/ipk/prod/navigation

/ipk/test/gabriel/face-follow
/ipk/test/colega/navigation-v2
```

Os módulos experimentais podem assinar os mesmos tópicos de sensores do T800, mas publicam seus resultados em tópicos próprios. Isso evita colisão de node names, portas web, arquivos temporários e outputs.

4.2 Command Gateway: uma única autoridade para comando físico

Nenhum módulo experimental deveria escrever diretamente no tópico final de movimento. Em vez disso, cada módulo publica uma intenção de comando. O Command Gateway mantém uma “control lease” (permissão temporária de controle) e decide qual origem pode comandar o hardware naquele momento.

```
TEST Gabriel  -> /ipk/test/gabriel/cmd_intent --\
TEST Colega   -> /ipk/test/colega/cmd_intent ----> Command Gateway -> tópico real do T800
PROD          -> /ipk/prod/cmd_intent ---------/
```

- Por padrão, PROD mantém a lease.
- Um teste pode solicitar a lease explicitamente pelo dashboard.
- O dashboard mostra claramente quem possui o controle.
- A lease expira automaticamente se o processo morrer, perder conexão ou ultrapassar timeout.
- Comandos e troca de controle ficam registrados em log.

4.3 Todos os tópicos ROS2, sem transmitir tudo o tempo inteiro

O Robot Agent deve descobrir e listar todos os tópicos, nós, serviços e actions. Porém, o backend remoto só recebe continuamente a telemetria que realmente precisa. Câmera, LiDAR, PointCloud e debug de alta frequência entram em modo sob demanda para evitar desperdício de banda e processamento.

Classe	Exemplos	Política
Telemetria contínua	bateria, temperatura, CPU, estado	Baixa frequência, sempre conectada
Estado ROS2	nodes, topics, services, actions	Descoberta periódica + eventos
Dados médios	joint states, IMU, diagnósticos	Assinatura configurável
Dados pesados	câmera, LiDAR, PointCloud	Somente quando solicitado
Comandos	movimento, restart, ações	Autorização + audit log + gateway

5. Como os serviços sobem e se recuperam

A recomendação inicial para um único robô é evitar Kubernetes. Ele adicionaria complexidade operacional sem necessidade. Para esta fase, Docker Compose + systemd oferece isolamento, versionamento, health checks e startup automático com uma curva de manutenção muito menor.

Inicialização proposta

```
T800 liga
  -> Ubuntu inicia
    -> systemd inicia t800-platform.service
      -> Docker Compose sobe infraestrutura IPK
        -> Robot Agent
        -> Telemetry
        -> Command Gateway
        -> módulos PROD
        -> health checks
      -> conexão segura com o Control Plane
      -> dashboard marca o robô como ONLINE
```

Políticas de recuperação

- Restart automático somente do serviço que falhou, não da plataforma inteira.
- Health check independente por módulo.
- Logs persistentes e identificados por versão/commit.
- Limites de CPU/memória para impedir que um teste consuma todos os recursos.
- Dependências explícitas: por exemplo, visão pode depender do Robot Agent, mas o dashboard não deve depender do reconhecimento facial.
- Serviços nativos da EngineAI não devem ser colocados dentro de containers ou reiniciados pela plataforma sem validação específica.

Estrutura de repositório sugerida

```
t800-platform/
├── apps/
│   ├── dashboard/
│   ├── backend/
│   └── robot-agent/
├── services/
│   ├── command-gateway/
│   ├── telemetry/
│   ├── vision/
│   ├── face-recognition/
│   └── navigation/
├── deploy/
│   ├── compose.prod.yml
│   ├── compose.test.yml
│   └── systemd/
├── configs/
├── scripts/
├── tests/
└── docs/
```

6. Fluxo de desenvolvimento e atualização

O código deve ter uma única fonte oficial em Git. Para a equipe atual, um monorepo facilita sincronizar versões de API, Robot Agent, módulos e documentação. Cada funcionalidade é desenvolvida em uma branch e nunca substitui diretamente a produção.

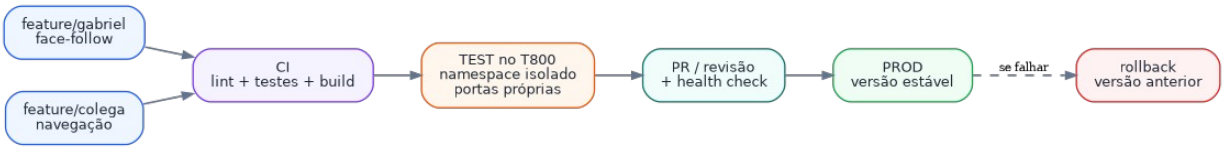


Figura 2 - Fluxo proposto de feature até produção.

Ambientes

Ambiente	Onde roda	Objetivo	Regra principal
DEV	Notebook	Desenvolvimento local, mocks, testes unitários	Não depende do T800 sempre
TEST	T800 em containers/namespace isolado	Validar com sensores e ROS2 reais	Não substitui PROD
PROD	T800 - versão oficial	Operação estável	Somente release aprovada

Deploy parcial

Cada módulo recebe uma imagem/versionamento próprio. Se houver mudança apenas em face-recognition, o deploy atualiza apenas esse serviço. Robot Agent, telemetria e navegação continuam executando a versão anterior.

```
face-recognition: 1.3.2 -> 1.4.0
telemetry:        1.1.0 -> permanece
command-gateway:  1.0.4 -> permanece
navigation:       0.9.8 -> permanece
```

Blue/Green simplificado para módulos críticos

Quando for necessário reduzir ainda mais o impacto, a nova versão pode subir ao lado da antiga em namespace separado. O health check valida a nova instância; somente depois o gateway/roteador troca o tráfego. Se falhar, a versão antiga continua disponível.

Rollback

- Cada release recebe tag imutável (ex.: v0.8.3) e commit correspondente.
- Manter localmente ou no registry pelo menos a última versão estável.
- Botão/ação de rollback deve trocar para a versão anterior sem editar código manualmente.
- Atualização de banco/configuração que não permita rollback precisa ser tratada como exceção e migrada de forma compatível.

7. Acesso remoto e segurança

7.1 Modelo recomendado de acesso

O T800 não precisa de IP público. Ele deve iniciar uma conexão segura para fora. Para administração, a equipe entra por uma rede privada virtual/overlay; para operação diária, usa o dashboard HTTPS.

```
Celular / notebook
   |
   +-- HTTPS + MFA -----------------> Dashboard / API
   |
   +-- VPN privada (admin) ---------> T800
                                      |
                                      +-- sem ROS2/DDS público
                                      +-- sem SSH aberto para a internet
```

7.2 Controles mínimos

Controle	Objetivo
Firewall default-deny	Bloquear entrada não necessária no T800. Apenas interfaces explicitamente autorizadas.
MFA para equipe	Evita que senha vazada, sozinha, conceda acesso ao dashboard.
RBAC	Separar visualização, operação, desenvolvimento e administração.
Identidade do robô	Robot Agent usa credencial/certificado próprio; não reutilizar senha humana.
TLS/mTLS	Criptografar dashboard, APIs e canal robot-backend.
Segredos fora do Git	Tokens, senhas e chaves não ficam hardcoded nem em commits.
Audit log	Registrar login, deploy, restart, alterações de configuração e comandos relevantes.
Rate limit e timeout	Reduz abuso e impede sessões/comandos esquecidos.
Imagens versionadas	Build reproduzível, dependências fixadas e origem controlada.
Atualização controlada	Atualizar dependências e base OS sem editar o robô de forma improvisada.

7.3 O que não fazer

- Não publicar portas DDS/ROS2 diretamente na internet.
- Não deixar Flask/FastAPI/debug server ouvindo publicamente sem autenticação.
- Não usar uma única conta root compartilhada entre desenvolvedores.
- Não manter tokens em arquivos versionados ou mensagens de chat.
- Não permitir que qualquer módulo de TEST publique diretamente no comando físico final.
- Não atualizar bibliotecas aleatoriamente no robô sem registrar versão e ter rollback.

8. Escopo do dashboard

O dashboard deve ser responsivo e útil tanto em notebook quanto em celular. A interface deve separar observação, operação e manutenção para evitar que ações administrativas fiquem misturadas com telemetria.

Área	Conteúdo
Overview	Online/offline, versão atual, uptime, CPU/RAM, bateria, temperatura, modo PROD/TEST.
Motors & Joints	Estado, temperaturas, falhas, posição/velocidade quando disponíveis.
ROS2 Explorer	Lista de nodes/topics/services/actions, tipo, frequência, publishers/subscribers e amostra de mensagem.
Sensors	IMU, LiDAR, câmera e diagnósticos; streaming pesado sob demanda.
Processes	Containers/serviços, health, versão, restart individual e logs.
TEST Sessions	Sessões de Gabriel/colega, branch/commit, namespace e controle da lease.
Deployments	Versão instalada, histórico, promover TEST -> PROD e rollback.
Logs	Busca por módulo, severidade, tempo e correlação com deploy.
Security	Sessões, usuários, papéis, tentativas de acesso e auditoria.
Settings	Configurações versionadas; alterações críticas exigem confirmação e log.

> **Ideia de UX importante**
> O dashboard deve mostrar permanentemente em qual contexto a equipe está atuando: PRODUÇÃO ou TESTE. Ações de TEST não podem parecer iguais às ações que afetam o robô oficial.

9. Stack inicial sugerida

A prioridade é manter a solução simples o suficiente para duas pessoas operarem. Abaixo está uma composição inicial; ela pode ser ajustada quando o dashboard atual for analisado.

Camada	Sugestão	Motivo
Frontend	React/Next.js ou frontend web equivalente	Responsivo, componentes, WebSocket, fácil deploy.
Backend	FastAPI (Python)	Boa integração com o ecossistema Python/ROS e APIs assíncronas.
Robot Agent	Python + rclpy	Integração direta com ROS2 Humble e introspecção do grafo.

Camada	Sugestão	Motivo
Tempo real	WebSocket seguro	Telemetria e eventos sem polling agressivo.
Banco	PostgreSQL	Usuários, robôs, deploys, auditoria e histórico.
Cache/eventos	Opcional no início	Redis/NATS/MQTT apenas se a necessidade surgir; evitar complexidade prematura.
Containers	Docker + Docker Compose	Isolamento, versionamento e deploy por módulo.
Startup	systemd	Garante que a stack suba com o robô e reinicie quando necessário.
Acesso admin	Tailscale/WireGuard	Rede privada sem publicar SSH/ROS2.
CI/CD	GitHub Actions ou GitLab CI	Build, testes, imagens e promoção de release.
Observabilidade	Logs estruturados + métricas	Pode começar simples e evoluir para Prometheus/Grafana se necessário.

> **Por que não Kubernetes agora?**
> Com um único T800, Docker Compose já resolve isolamento, versionamento, health checks, restart e perfis de ambiente. Kubernetes pode ser avaliado se a frota crescer ou se houver necessidade real de orquestração distribuída.

10. Plano de implementação por fases

Fase 0 - Inventário - Mapear o dashboard atual, processos do T800, nós ROS2, tópicos, portas, scripts, dependências e serviços EngineAI. Resultado: diagrama do “como está”.

Fase 1 - Base do robô - Criar repositório oficial, Robot Agent, systemd, Docker Compose, health checks e logs. Sem mudar ainda funcionalidades de alto nível.

Fase 2 - Dashboard remoto - Backend + frontend com autenticação; telemetria, status, bateria, temperaturas e ROS2 Explorer.

Fase 3 - TEST paralelo - Namespaces, compose de teste por desenvolvedor, limites de recurso e painel de sessões.

Fase 4 - Command Gateway - Introduzir lease e roteamento único para comandos físicos. Remover publicação direta de módulos experimentais.

Fase 5 - CI/CD - Build automático, imagens versionadas, teste, promoção para produção e rollback.

Fase 6 - Hardening - MFA, RBAC, mTLS, firewall, segredo centralizado, auditoria, backup e revisão de permissões.

Fase 7 - Evolução - Histórico de métricas, alertas, vídeo otimizado, múltiplos T800, gerenciamento de frota e observabilidade avançada.

Primeiro marco útil

> **MVP técnico recomendado**
>
> Antes de reconstruir toda a interface: colocar no T800 um Robot Agent + stack Compose iniciada por systemd + acesso privado remoto + endpoint de health. Depois conectar o dashboard. Isso cria a fundação que impede o projeto de voltar ao modelo “rodar app manualmente no computador de alguém”.

11. Critérios de sucesso

- Após reiniciar o T800, a plataforma volta sozinha sem terminal manual.
- Dashboard pode ser aberto de um celular fora da rede local, por usuário autorizado.
- Nenhuma porta ROS2/DDS precisa estar exposta publicamente.
- Equipe consegue visualizar todos os nodes/topics/services/actions do ROS2 Humble.
- Gabriel e outro desenvolvedor conseguem executar versões TEST diferentes sem substituir PROD.
- Reiniciar ou atualizar um módulo não derruba os demais módulos independentes.
- É possível identificar exatamente qual commit/versão está rodando em cada serviço.
- Uma atualização com falha pode voltar para a versão anterior com procedimento curto e documentado.
- Ações administrativas e comandos relevantes ficam em audit log.
- Um novo desenvolvedor consegue levantar o ambiente lendo README + documentação, sem depender de conhecimento informal.

12. Documentação mínima do projeto

```
docs/
├── ARCHITECTURE.md      # desenho e decisões
├── INSTALL_ROBOT.md     # preparação do T800
├── DEVELOPMENT.md       # DEV e TEST
├── DEPLOYMENT.md        # release e rollback
├── ROS2.md              # nós, tópicos, namespaces e gateway
├── SECURITY.md          # acessos, credenciais e hardening
├── TROUBLESHOOTING.md   # falhas comuns
└── RUNBOOK.md           # procedimentos operacionais
```

Conclusão

A mudança mais importante não é trocar a aparência do dashboard: é desacoplar a plataforma. O T800 deve hospedar serviços independentes, com uma camada própria para ROS2, um gateway único para comandos físicos e uma separação explícita entre TEST e PROD. O dashboard passa a administrar essa plataforma, e não a concentrar toda a lógica do robô em um único processo.

> **Próximo passo**
>
> Quando o projeto atual do dashboard for disponibilizado, o primeiro trabalho deve ser um inventário técnico do código existente: o que permanece, o que vira módulo, o que deve ser substituído e quais pontos atualmente causam colisões entre processos. A partir desse mapa, esta arquitetura pode virar um plano de migração incremental, sem reescrever tudo de uma vez.

Fim - Proposta de Arquitetura T800 / v0.1